RetailIQ: An End-to-End E-Commerce Customer and Sales Analytics Platform

A Project Report Submitted in

the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

B. Kundana Mani : 2420030558

M. Sai Kridhay Kumar : 2420030211

Under the Esteemed Guidance of

Dr. N. Sunanda

Associate Professor

Department of Computer Science and Engineering

AY: 2026-2027

Semester: Odd Semester

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

The Project Report entitled "**RetailIQ: An End-to-End E-Commerce Customer and Sales Analytics Platform**" is a record of Bonafide work of **B. Kundana Mani – 2420030558, M. Sai Kridhay Kumar – 2420030211** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

B. Kundana Mani – 2420030558

M. Sai Kridhay Kumar – 2420030211

CERTIFICATE

This is to certify that the project report entitled " **RetailIQ: An End-to-End E-Commerce Customer and Sales Analytics Platform** " is a bonafide work carried out and submitted by B. Kundana Mani (2420030558), M. Sai Kridhay Kumar (2420030211) in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science and Engineering, K L (Deemed to be) University, during the academic year 2026–2027.

Signature of the Guide
Dr. N. Sunanda
Associate Professor, CSE

Course Coordinator
Dr. N. Sunanda
Department of CSE

Head of the Department
Dr. P. Venkateswara Rao
Department of CSE

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is certify that the mini project-based report entitled "**RetailIQ: An End-to-End E-Commerce Customer and Sales Analytics Platform**" is a bonafide work done and submitted by **B. Kundana Mani – 2420030558, M. Sai Kridhay Kumar – 2420030211** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide
Dr. N. Sunanda
Associate Professor, CSE

Signature of the Course Coordinator
Dr. N. Sunanda
Department of CSE

Signature of the Head of the Department
Dr. P. Venkateswara Rao
Department of CSE

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator and guide **Dr. N. Sunanda,** Department of Computer Science and Engineering which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **P. Venkateshwara Rao**, Head of the ***Department of Computer Science and Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

B. Kundana Mani – 2420030558

M. Sai Kridhay Kumar – 2420030211

Abstract

E-commerce platforms generate large volumes of customer, order, product, payment, return and review data, which makes it difficult to obtain a unified view of business performance and customer behaviour. This report presents RetailIQ, an end-to-end e-commerce customer and sales analytics platform that integrates data preprocessing, feature engineering, exploratory data analysis, customer analysis, product performance analysis, sales and revenue analysis, customer segmentation and interactive visualization in a single workflow.

The system uses a simulated Indian e-commerce dataset of seven relational tables: 25,000 customers, 1,00,000 orders, 2,54,331 order items, 550 products, 1,00,000 payments, 12,075 returns and 77,530 reviews. The data is cleaned and integrated into a master table of 2,54,331 rows, and 15 temporal, financial, customer, product, discount and return features are engineered. Customer behaviour is modelled with Recency, Frequency and Monetary (RFM) scoring, which places 22,820 purchasing customers into six actionable segments, and with K-Means clustering on standardised RFM features, where the silhouette coefficient selects K = 2 (s = 0.4378).

The analysis reveals that Electronics leads revenue (₹122.4 Cr) at only a 0.69% margin while Fashion is the most profitable category (33.1% margin), that 33 products are loss-making, that the overall return rate is 12.07% with size and damage issues as the main causes, that UPI is the dominant payment method, and that 59.7% of reviews are positive. All results are delivered through a seven-page interactive Streamlit dashboard with Plotly charts, giving executives, marketing, product, operations and finance teams a centralized, data-driven view of the business.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience 1.7 Literature survey & research gap	1-5
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	6-7
3	Technology Stack	3.1 Front-end (Streamlit, Plotly, CSS) 3.2 Back-end (Python analytics pipeline) 3.3 Database (file-based CSV data store) 3.4 Version control (Git, VS Code) 3.5 APIs and external services	8-10
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (local and cloud)	11-13
5	Design	5.1 Wireframes or mockups 5.2 ER Diagram / Database schema	14-17
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 Backend logic (pipeline and business logic) 6.4 Database connectivity 6.5 API integration 6.6 Authentication & Authorization	18-30
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	31-37
8	Testing	8.1 Functional testing (pipeline and dashboard) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	38-39
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	40-41
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	42-43
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	44
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	45-46
13	References	- Books, tutorials, APIs, documentation sites used	
14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide - Certifications / publication details	

CHAPTER 1

Introduction

1.1 Background of the Project

The rapid growth of online retail in India has produced very large volumes of operational data. Every purchase leaves a trail across several systems: the customer profile, the order header, the individual order lines, the product catalogue, the payment gateway, the returns desk and the review platform. Each of these sources holds valuable information about purchasing behaviour, product performance, sales trends and revenue generation, yet they are usually stored and analysed in isolation.

Analysing these sources one at a time makes it difficult to see the business as a whole. A category may look healthy on revenue but be barely profitable; a customer may look valuable on total spend but has not ordered in two years; a product may sell well but generate a disproportionate share of returns. These insights only appear when the data is joined, cleaned and examined together.

RetailIQ is an end-to-end e-commerce customer and sales analytics platform built to close this gap. It ingests seven relational datasets from a simulated Indian e-commerce marketplace (25,000 customers, 1,00,000 orders, 2,54,331 order lines, 550 products, 1,00,000 payments, 12,075 returns and 77,530 reviews), prepares them through a reproducible Python pipeline, applies descriptive analytics, RFM scoring and K-Means clustering, and presents the results in a seven-page interactive Streamlit dashboard.

1.2 Problem Statement

E-commerce businesses generate large volumes of transaction data, but raw data is difficult to interpret and use for decision-making. Specifically:

- Customer purchasing patterns are difficult to identify when customer, order and item data sit in separate tables.
- Product and category performance cannot easily be compared on revenue, profit, margin and rating at the same time.
- Sales and revenue trends across months, weekdays and states remain hidden inside hundreds of thousands of rows.

- Identifying valuable and at-risk customer segments requires systematic scoring and clustering, not manual inspection.
- Raw data contains missing values, inconsistent types and text dates that must be resolved before analysis.
- Manual spreadsheet analysis is slow, error-prone and does not give managers a single, centralized view of key metrics.

1.3 Scope of the Project

The scope of RetailIQ covers the complete analytics life-cycle on a static snapshot of marketplace data:

Table 1.1: *Scope boundaries of RetailIQ*

In scope	Out of scope
Ingestion of 7 CSV tables (customers, orders, order items, products, payments, returns, reviews)	Real-time streaming ingestion from a live store
Data inspection, profiling, quality checks and cleaning	Writing back to an operational database
Integration into one master analytical table and feature engineering	Supervised prediction (churn, CLV forecasting)
EDA, RFM segmentation, K-Means clustering, product, geography, returns, payments and review analysis	Transformer-based NLP on free-text reviews
Seven-page interactive Streamlit dashboard for business users	User login, roles and multi-tenant access control

1.4 Objectives

1. **Customer Analysis** – understand customer purchasing behaviour and spending patterns.
2. **Product Analysis** – identify top-performing and low-performing products and categories using revenue, profit, margin and rating.
3. **Sales Analysis** – analyse sales trends across months, quarters and days of the week.
4. **Revenue Analysis** – identify the major contributors to overall revenue across categories, brands and Indian states.
5. **Customer Segmentation** – group customers using Recency, Frequency and Monetary (RFM) scoring and K-Means clustering.

6. **Dashboard Development** – present key performance indicators and insights through an interactive, multi-page dashboard.

The study is guided by four research questions: (RQ1) how heterogeneous e-commerce data can be preprocessed and integrated for comprehensive analysis; (RQ2) which purchasing patterns and segments emerge from RFM analysis and clustering; (RQ3) how product, category, sales, revenue and geographical data reveal performance patterns; and (RQ4) how an interactive dashboard can present these insights in one place.

1.5 Importance of the Application

RetailIQ turns raw transactions into decisions. A few examples from the results illustrate why an integrated view matters:

- **Revenue is not profit.** Electronics generates ₹122.4 Cr of revenue but only a 0.69% margin, while Fashion earns the highest profit (₹4.6 Cr) at a 33.1% margin. Looking at revenue alone would steer promotions to the wrong category.
- **Customer value is concentrated.** Champions and Loyal Customers form 38% of buyers, while 23.7% of buyers are At Risk or High Value At Risk – a clear target list for win-back campaigns.
- **Returns have fixable causes.** Size issues and damaged products together explain 34.2% of all 12,075 returns, pointing to better size charts and packaging.
- **33 products sell at a loss**, flagged automatically for pricing or sourcing review.

Because the platform is built entirely on open-source Python tools, it can be adopted by small and medium online sellers at negligible cost.

1.6 Target Users / Audience

Table 1.2: *Target users of RetailIQ*

User group	What they use RetailIQ for	Main dashboard page
Executive leadership	Overall revenue, profit, margin, orders and return rate at a glance	Executive Overview
Marketing & CRM teams	RFM segments, K-Means clusters, retention and win-back targeting	Customer Intelligence
Merchandising / product managers	Category and brand profitability, loss-making SKUs	Product Intelligence
Regional / sales managers	State and city performance, regional margins	India Analytics

1.7 Literature Survey and Research Gap

Prior work on retail analytics has mostly addressed one analytical dimension at a time. Table 1.3 summarises representative studies reviewed during the project.

Table 1.3: *Literature survey summary*

Authors	Title / focus	Methodology	Inference	Limitation
Chen et al. (2012)	Data mining for the online retail industry	RFM, K-Means	Effective customer segmentation	Mainly customer-focused
Hong & Kim (2012)	Customer segmentation in online stores	SOM, K-Means, KNN	Identifies customer groups	Limited business-wide analysis
Griva et al. (2018)	Retail business analytics	Market basket, clustering	Identifies shopping patterns	Focuses on customer visits
Wei (2021)	Segmentation using RFM and K-Means	RFM + K-Means	Better granularity than rule cut-offs	Segmentation only
Wong et al. (2024)	Customer segmentation in e-commerce	RFM, clustering	Identifies valuable groups	Segmentation-focused
Few (2013); Chen & Zhang (2018)	Dashboard design; visual analytics	Visual design principles	Interactive filtering speeds decisions	No analytics pipeline

Identified gap

Most approaches focus on a specific aspect – segmentation, RFM, basket analysis or dashboard design – rather than a unified view of the complete business. RetailIQ integrates **customer behaviour + product performance + sales + revenue + operations** in one reproducible workflow and one dashboard.

CHAPTER 2

System Requirements

RetailIQ is a data-intensive but lightweight application. The pipeline processes about 46 MB of raw CSV data and writes roughly 306 MB of intermediate and analytical outputs; the dashboard then serves these outputs to a web browser.

2.1 Hardware Requirements

Table 2.1: *Hardware requirements*

Component	Minimum	Used / recommended
Processor	Dual-core x86_64, 2.0 GHz	Multi-core x86_64, 2.50 GHz
RAM	8 GB	16 GB (feature table of 2,54,331 × 69 loads in memory)
Storage	1 GB free	SSD with 2 GB free (raw data + outputs + environment)
Display	1366 × 768	1920 × 1080 for the wide dashboard layout
Network	Not required for analysis	Only for installing packages and loading web fonts

2.2 Software Requirements

Table 2.2: *Software requirements*

Software	Version	Purpose
Operating system	Windows 10/11 64-bit (also runs on Linux / macOS)	Host environment
Python	3.13	Programming language for pipeline and dashboard
Pandas	2.2.3	Loading, cleaning, merging and aggregating tabular data
NumPy	2.1.2	Vectorised numerical computation
scikit-learn	1.6.1	StandardScaler, KMeans, silhouette_score
Matplotlib	3.10.0	Static EDA charts saved as PNG
Plotly	6.0.0	Interactive charts in the dashboard
Streamlit	1.42.0	Multi-page web dashboard and local web server
Web browser	Chrome / Edge / Firefox (current)	Viewing the dashboard

2.3 Development Tools and Frameworks

- **Visual Studio Code** – primary editor, configured with the Python extension and the system interpreter (`python-envs.defaultEnvManager`).
- **Python virtual environment / pip** – dependency installation and isolation.
- **Streamlit framework** – turns a Python script into a reactive web app; each widget interaction re-runs the script.
- **Plotly Express and Graph Objects** – declarative interactive charts with hover, zoom and export.
- **Matplotlib** – reproducible static charts for the EDA and model-selection outputs.
- **Git** – version control of scripts, dashboard and documentation.
- **Kaggle** – source of the simulated *E-Commerce Sales and Customer Analytics* dataset.

CHAPTER 3

Technology Stack

RetailIQ follows a pure-Python data stack. Instead of the classic web split of a JavaScript front-end, a REST back-end and a SQL database, the front-end is a Streamlit application, the back-end is a set of Python analytics modules, and the data store is a structured folder of CSV files.

Table 3.1: *RetailIQ technology stack*

Layer	Technology	Role in RetailIQ
Front-end	Streamlit, Plotly, custom CSS (Inter / Space Grotesk fonts)	Seven-page dark-theme dashboard, KPI cards, interactive charts, filters
Back-end	Python 3.13, Pandas, NumPy, scikit-learn, Matplotlib	12 pipeline scripts: inspection, cleaning, integration, features, analytics, clustering
Data store	CSV files (datasets/ and outputs/)	Raw tables, cleaned tables, master table, features and analytic summaries
Version control	Git, VS Code	Source tracking and collaboration
External services	Kaggle dataset, Google Fonts	Data source and dashboard typography

3.1 Front-end: Streamlit, Plotly and Custom CSS

Streamlit renders the user interface directly from Python. The sidebar uses `st.radio` for navigation between seven pages, `st.columns` to lay out KPI tiles, `st.metric` for headline numbers, `st.date_input` and `st.multiselect` for filters, and `st.dataframe` for tables. A design-system dictionary (`COLORS`) and an injected style sheet give the app a consistent dark theme with an indigo accent (#6c7bff) and teal secondary (#38bfa7).

Plotly provides every chart: area and line charts for revenue trends, horizontal bar charts for rankings, donut charts for status shares, and scatter plots for clusters and state revenue versus profit. A shared `style_chart()` helper applies transparent backgrounds, fonts and grid colours so all 30+ charts look alike.

3.2 Back-end: Python Analytics Pipeline

The back-end is a batch pipeline of independent Python scripts in `src/`. Each script reads the previous stage's CSV output, performs one responsibility and writes its own output folder, which makes every stage individually re-runnable and testable.

- **Pandas** – `read_csv`, `to_datetime`, `fillna`, `merge`, `groupby().agg()`, `qcut` for quintile scoring.
- **NumPy** – vectorised arithmetic and handling of infinite values in margin calculations.
- **scikit-learn** – `StandardScaler` for z-score normalisation, `KMeans` (random_state = 42, n_init = 10) and `silhouette_score` for model selection.
- **Matplotlib** – EDA charts saved to `outputs/eda/` and the silhouette curve saved to `outputs/customer_analysis/`.

3.3 Version Control: Git and VS Code

Source code, the dashboard and documentation are tracked with Git. Large generated CSV outputs are reproducible from the scripts and can be excluded through `.gitignore`, keeping the repository small. VS Code's workspace settings (`.vscode/settings.json`) pin the Python environment so that all team members run the same interpreter.

3.4 APIs and External Services

- **Kaggle** – the simulated Indian e-commerce dataset (7 CSV files) is the sole data source.
- **Google Fonts API** – the dashboard imports the *Inter* and *Space Grotesk* typefaces through a CSS `@import`.
- **Library APIs** – Streamlit's `@st.cache_data` caching API, Plotly's figure API and scikit-learn's estimator API (`fit`, `fit_predict`) form the programmatic interfaces between modules.
- No payment, authentication or cloud-database service is required, which keeps operating cost at zero.

CHAPTER 4

System Architecture

4.1 High-level Architecture Diagram

RetailIQ is organised as five stacked layers. Data flows strictly downward from the raw sources to the business stakeholders, and every layer communicates with the next only through files.

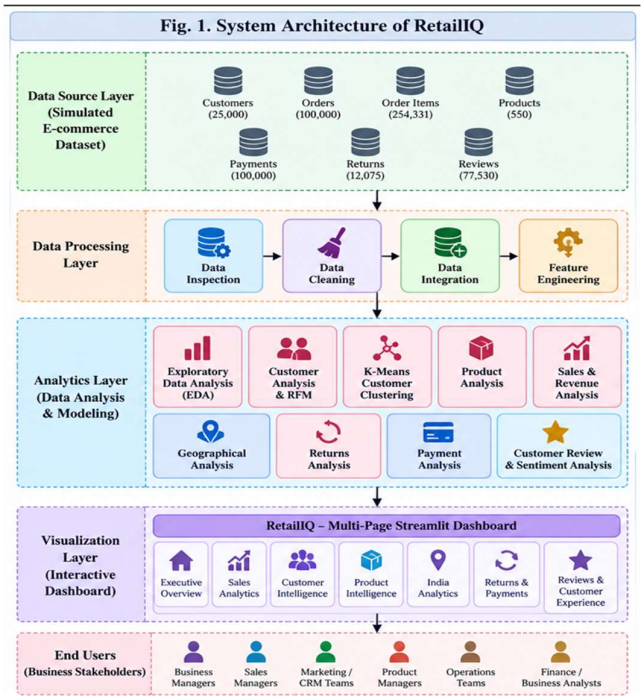


Figure 4.1: *Layered system architecture of RetailIQ*

4.2 Description of Each Layer

Table 4.1: *Responsibilities of the architectural layers*

Layer	Components	Responsibility
1. Data Source Layer	customers, orders, order_items, products, payments, returns, reviews (CSV)	Holds the raw simulated marketplace data
2. Data Processing Layer	Data inspection, data cleaning, data integration, feature engineering	Makes data correct, consistent and analysis-ready
3. Analytics Layer	EDA, RFM, K-Means, product, sales & revenue, geography, returns, payments, review sentiment	Produces metrics, segments, clusters and summaries
4. Visualization Layer	Streamlit multi-page dashboard with Plotly charts	Presents KPIs and insights interactively
5. End Users	Business, sales, marketing, product, operations and finance teams	Consume insights and act on them

Front-end (visualization layer). `dashboard/app.py` loads all analytical outputs once through a cached `load_data()` function, derives a few page-level aggregates on the fly (for example the date-filtered sales view), and renders each page with shared helpers for headers, KPI cards, charts and insight cards.

Back-end (processing and analytics layers). Twelve scripts in `src/` run in a fixed order. They are stateless: given the same inputs they always produce the same outputs, which is guaranteed by fixed random seeds in clustering.

Data layer. The CSV store described in Section 3.3 acts as the persistence layer and as the contract between back-end and front-end.

4.3 Deployment Architecture

The current deployment is local: the pipeline and Streamlit server run on one workstation, and users open the dashboard in a browser at `http://localhost:8501`. Streamlit communicates with the browser over HTTP and a WebSocket, re-executing the script whenever a widget changes. The same structure can be containerised with Docker and hosted on Streamlit Community Cloud, AWS or GCP with a GitHub Actions pipeline re-running the analytics when new data arrives (see Chapter 9).

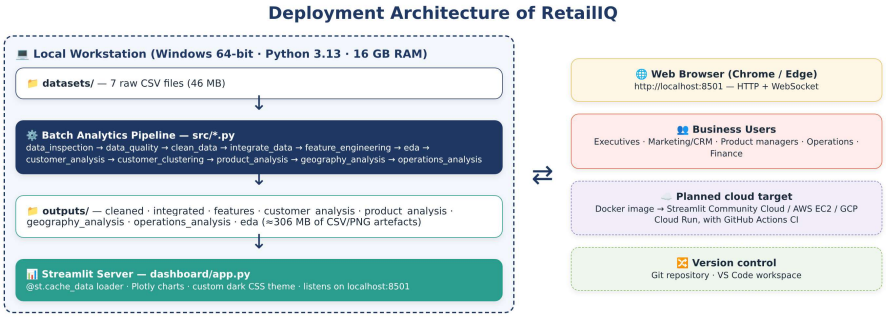


Figure 4.3: *Deployment architecture – current local setup and planned cloud target*

CHAPTER 5

Design

5.1 Wireframes and Mockups

Implementation workflow design

The analytical workflow was designed as ten sequential stages, from data collection to dashboard development. Each stage maps to one or more scripts in the code base.

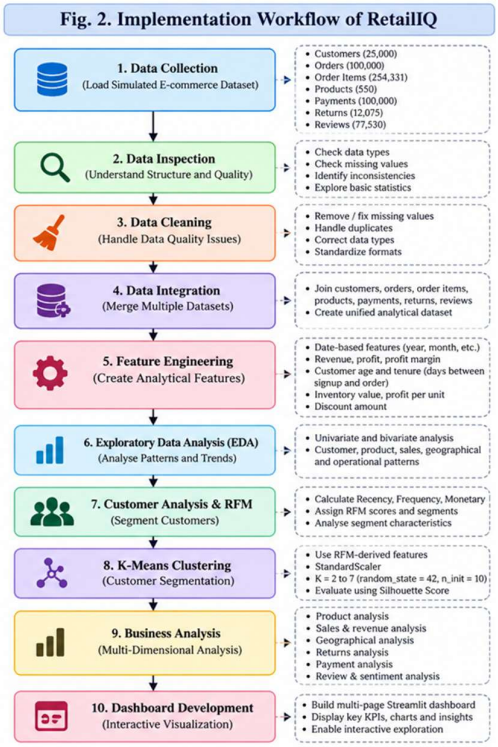


Figure 5.3: *Ten-stage implementation workflow of RetailIQ*

5.2 ER Diagram / Database Schema

The seven source tables form a star-like relational model centred on ORDERS. Customers place orders; each order contains one or more order items, each referring to one product; each order has a payment record and may have returns; reviews link a customer, an order and a product.

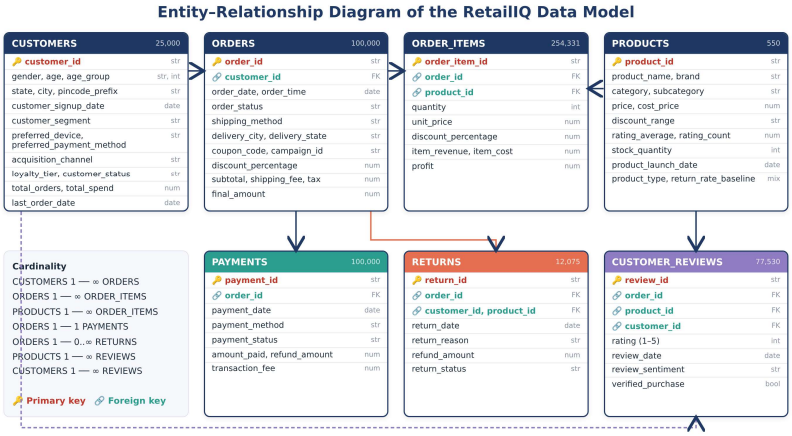


Figure 5.4: *Entity–relationship diagram of the RetailIQ data model*

Table 5.1: *Database schema summary*

Table	Records	Primary key	Foreign keys	Key attributes
customers	25,000	customer_id	–	age, gender, state, city, signup date, loyalty tier, status, device
orders	1,00,000	order_id	customer_id	order date/time, status, shipping, coupon, campaign, final amount
order_items	2,54,331	order_item_id	order_id, product_id	quantity, unit price, discount %, item revenue, cost, profit
products	550	product_id	–	name, category (22), subcategory, brand (148), price, cost, stock, rating
payments	1,00,000	payment_id	order_id	method, status, amount paid, fee, refund amount
returns	12,075	return_id	order_id, customer_id, product_id	return date, reason, refund amount, status
customer_reviews	77,530	review_id	order_id, customer_id, product_id	rating 1–5, sentiment, verified purchase

CHAPTER 6

Implementation

6.1 Module-wise Implementation

6.1.1 Data inspection, profiling and quality checks

`data_inspection.py` prints shapes, data types and sample rows; `profile_data.py` profiles every column; `data_quality.py` checks referential integrity (for example, order items whose order or product does not exist) and lists the distinct values of status columns. The scan found missing values in seven columns and **no orphan foreign keys**.

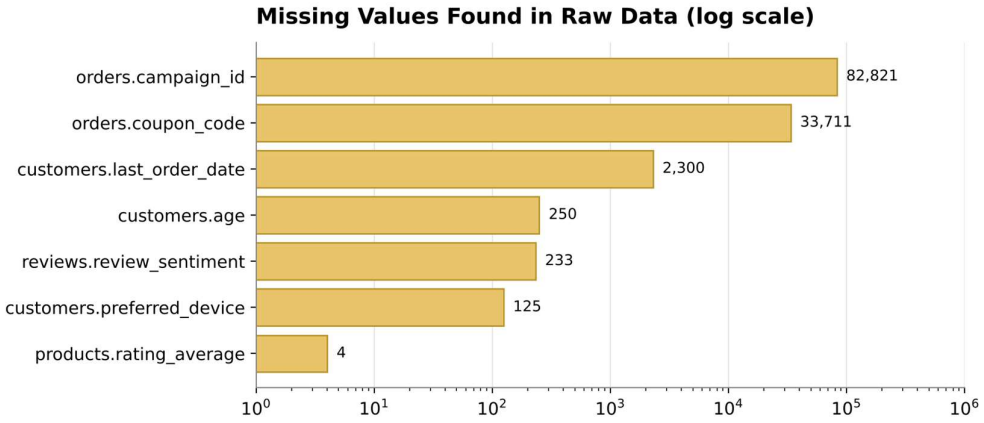


Figure 6.2: *Missing values detected in the raw data*

6.1.2 Data cleaning

`clean_data.py` standardises all date columns with `pd.to_datetime` and treats each missing-value pattern according to its meaning:

Listing 6.1 – sentiment imputation in clean_data.py

```
# Derive missing sentiment from rating
def get_sentiment(rating):
    if rating >= 4:
        return "Positive"
    elif rating == 3:
        return "Neutral"
    else:
        return "Negative"

reviews["review_sentiment"] = reviews["review_sentiment"].fillna(
    reviews["rating"].apply(get_sentiment))
```

6.1.3 Data integration

`integrate_data.py` builds the master analytical table with three left joins: orders ⟶ customers on `customer_id`, then ⟶ order_items on `order_id`, then ⟶ products on `product_id`. The result has one row per order line: **2,54,331 rows × 54 columns**. Returns are attached later as a flag to avoid one-to-many row duplication.

Listing 6.2 – four-way merge

```
master = orders.merge(customers, on="customer_id", how="left", suffixes=("", "_customer"))
master = master.merge(order_items, on="order_id", how="left", suffixes=("", "_item"))
master = master.merge(products, on="product_id", how="left", suffixes=("", "_product"))
master.to_csv("outputs/integrated/retailiq_master.csv", index=False)
```

6.1.4 Feature engineering

`feature_engineering.py` adds 15 derived attributes, bringing the table to 69 columns:

Table 6.2: *Engineered features*

Group	Features	Formula / rule
Temporal	order_year, order_month, order_month_name, order_quarter, order_day, order_day_name	Extracted from order_date
Financial	revenue, item_profit, profit_margin	profit_margin = item_profit / revenue × 100 (±∞ → 0)
Customer	customer_age, customer_lifetime_days	order_date − signup_date, clipped at 0
Product	inventory_value, profit_per_unit	stock × cost_price; price − cost_price
Discount	discount_amount	unit_price × quantity × discount % / 100
Returns	has_return	1 if order_id appears in returns, else 0

The feature step shows that total discounts given amount to **₹36.29 Cr** against gross item revenue of **₹244.99 Cr**, and the median customer places an order 106 days after signing up.

6.1.5 Exploratory data analysis

`eda.py` produces 12 charts covering revenue and profit by month and category, order status, customer status, loyalty tiers, payment methods, ratings, return reasons, states and top products. Revenue was roughly flat through 2023–2024 (₹66.1 Cr and ₹65.2 Cr) and rose sharply in 2025 to ₹113.7 Cr, with a pronounced peak in the final quarter. Weekends earn about 25% more revenue per day than weekdays.

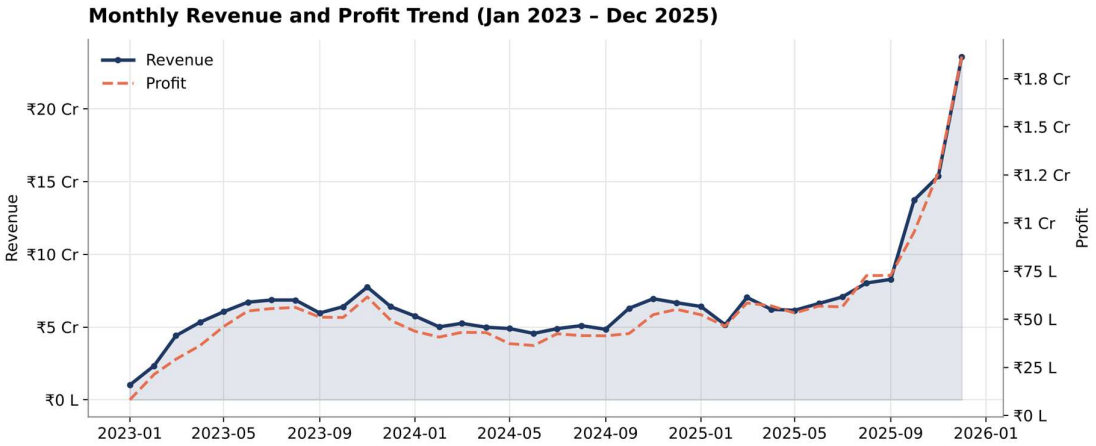


Figure 6.3: *Monthly revenue and profit, January 2023 – December 2025*

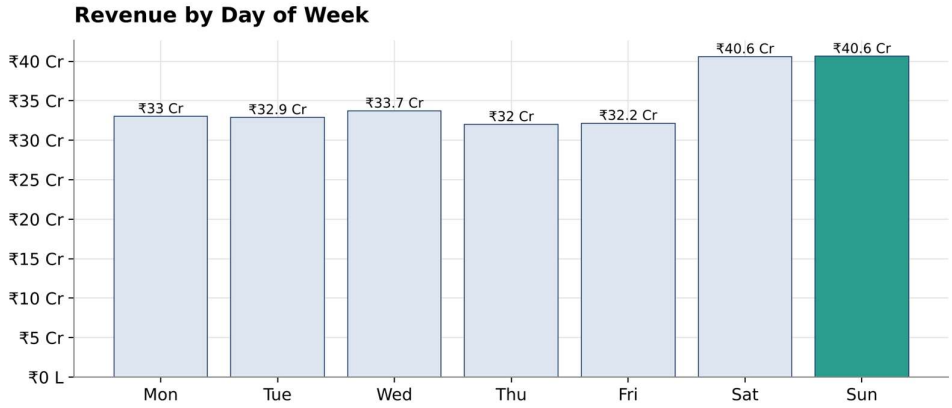


Figure 6.4: *Revenue by day of the week – Saturday and Sunday lead*

Order status is dominated by deliveries (80.7%); 8.1% of orders are cancelled, 6.7% returned, 2.9% failed and 1.7% still processing. Among the 25,000 registered customers, 13,365 are marked Churned, 7,234 Active, 2,101 At Risk and 2,300 Never Purchased; the loyalty base is Bronze-heavy (13,650 Bronze, 6,810 Silver, 3,405 Gold, 1,135 Platinum).

6.1.6 Customer analysis and RFM segmentation

`customer_analysis.py` aggregates the feature table per customer (22,820 purchasing customers) and computes Recency (days since last order, relative to the latest order date), Frequency (distinct orders) and Monetary value (total revenue). Each is scored 1–5 by quintiles with `pd.qcut`; recency is reversed so that recent buyers score high. Rule-based logic then assigns six business segments.

Table 6.3: *RFM segment profile (22,820 customers)*

RFM segment	Customers	Share	Avg recency (days)	Avg orders	Avg spend (₹)
Regular Customers	5,693	24.9%	575.5	1.6	25,575
Loyal Customers	5,628	24.7%	160.7	5.1	1,05,797
At Risk	4,945	21.7%	710.4	6.0	1,51,822
Champions	3,119	13.7%	41.0	8.4	2,47,100
New Customers	2,967	13.0%	57.4	1.8	38,930
High Value At Risk	468	2.1%	732.9	1.9	1,53,651

6.1.7 K-Means customer clustering

`customer_clustering.py` complements the rule-based segments with a data-driven grouping. Recency, frequency and monetary values are standardised with `StandardScaler` (so that rupee amounts in lakhs do not dominate order counts), and K-Means models are trained for K = 2 to 7. The model with the highest mean silhouette coefficient is selected:

$$s(i) = [\, b(i) - a(i) \,] \,/\, \max\{\, a(i), b(i) \,\}$$

where a(i) is the mean distance from sample i to the other points in its cluster and b(i) is the mean distance to the nearest other cluster.

Listing 6.4 – choosing K by silhouette score

```
X_scaled = StandardScaler().fit_transform(df[["recency", "frequency", "monetary"]])
for k in range(2, 8):
    model  = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = model.fit_predict(X_scaled)
    silhouette_scores.append(silhouette_score(X_scaled, labels))
best_k = list(range(2, 8))[silhouette_scores.index(max(silhouette_scores))]
```

Table 6.4: *Silhouette coefficient for K = 2 … 7*

K	2	3	4	5	6	7
Silhouette	**0.4378**	0.4107	0.3983	0.4013	0.3338	0.3419

6.1.8 Product and category analysis

`product_analysis.py` aggregates quantity, revenue, profit, rating, rating count and stock per product, category (22) and brand (148), and computes profit margin. The key finding is the gap between revenue and profitability.

Table 6.6: *Performance of the top revenue categories*

Category	Units	Revenue (₹ Cr)	Profit (₹ Cr)	Margin	Avg rating
Electronics	59,068	122.36	0.85	0.69%	3.97
Appliances	35,842	40.52	1.57	3.89%	4.00
Home & Kitchen	60,205	15.81	2.84	17.96%	3.92
Furniture	8,118	14.70	1.43	9.76%	3.87
Fashion	98,750	13.94	4.62	33.14%	4.01
Footwear	32,501	8.26	2.43	29.42%	4.04
Bags	29,948	5.59	1.60	28.59%	4.20

At product level, **33 of the 550 products have negative total profit**. These are listed on the Product Intelligence page for pricing or sourcing review.

6.1.9 Geographical analysis

`geography_analysis.py` aggregates orders, customers, quantity, revenue, profit and margin by delivery state and city. Maharashtra leads with ₹34.27 Cr revenue from 14,196 orders, followed by Uttar Pradesh (₹26.64 Cr) and Gujarat (₹25.95 Cr). State margins are tightly clustered between roughly 7.5% and 8.8%, so regional differences are driven by volume rather than profitability.

6.1.10 Operations: returns, payments and reviews

`operations_analysis.py` summarises returns by reason, status and category, payments by method and status, and reviews by rating, sentiment, category and verification. Of 12,075 returns, 8,473 refunds are completed, 1,792 approved, 938 rejected and 872 pending; Fashion carries the most returns (3,185).

UPI is the dominant payment method (31,971 transactions, ₹78.06 Cr), followed by Cash on Delivery (17,274) and credit cards (15,063). 82.1% of payments succeed, 10.7% are refunded and 5.1% fail.

Customer reviews are mostly favourable: 59.7% positive, 31.2% neutral and 9.0% negative; 97.1% (75,296) of reviews come from verified purchases. Pet Supplies, Toys and Watches have the highest average category ratings.

6.2 Front-end Logic (UI Rendering and State Management)

The dashboard is a single Streamlit script structured as **design system → data loading → helpers → sidebar → page branches**. Streamlit re-runs the script from top to bottom on every interaction; the selected page is held in the `st.radio` widget state, and filter values (date range, categories) are held in their widgets' state. Expensive work is avoided on re-runs because all CSV loading is cached.

Listing 6.5 – page routing and filter state in app.py

```
page = st.radio("Navigation", [
    "Executive Overview", "Sales Analytics", "Customer Intelligence",
    "Product Intelligence", "India Analytics", "Returns & Payments",
    "Reviews & Customer Experience"])

if page == "Executive Overview":
    page_header("Executive Overview", "Marketplace performance, revenue trends, and key
business signals at a glance.")
    ...
elif page == "Sales Analytics":
    date_range = st.date_input("Analysis Period", [date_min, date_max])
    filtered = df[(df["date"] >= start) & (df["date"] <= end)]
```

Reusable rendering helpers keep the pages consistent: `kpi_card()` returns styled HTML for headline tiles, `style_chart()` applies the common Plotly theme, `page_header()` and `section_heading()` print titles, and `insight_card()` renders narrative call-outs. Indian number formatting is centralised in `money()`.

6.3 Backend Logic (Pipeline and Business Logic)

Because RetailIQ is an analytics product rather than a transactional web service, its "routes and controllers" are the ordered pipeline stages, and its business logic is the set of formulas and rules applied to the data:

Table 6.7: *Business logic implemented in the back-end*

Business rule	Implementation
Profit margin	item_profit / revenue × 100; infinite values replaced by 0
Return rate	returns / orders × 100 = 12,075 / 1,00,000 = 12.07%
Recency reference date	Latest order date in the dataset (31 Dec 2025)
RFM scores	Quintiles (1–5); recency reversed; frequency and monetary ranked first to break ties
Segment rules	Champions → Loyal → New → At Risk → High Value At Risk → Regular (first match wins)
Clustering	StandardScaler + KMeans(random_state = 42, n_init = 10); K chosen by max silhouette

Business rule	Implementation
Sentiment	Rating ≥ 4 Positive, = 3 Neutral, ≤ 2 Negative
Loss-making product	total_profit < 0 across all of its order lines

6.4 Database Connectivity

Connectivity between stages is file-based. Each script reads its inputs with `pd.read_csv` and writes results with `to_csv`, creating folders with `os.makedirs(..., exist_ok=True)`. The dashboard connects to the store through one cached loader that reads 18 analytical files into a dictionary of DataFrames.

Caching means the 144 MB feature table is parsed once per server session; subsequent page switches and filter changes are served from memory in well under a second.

6.5 API Integration

- **scikit-learn estimator API** – `StandardScaler().fit_transform()`, `KMeans().fit_predict()`, `silhouette_score()`.
- **Plotly figure API** – `px.area`, `px.bar`, `px.pie(hole=0.65)`, `px.scatter` and `go.Figure` objects passed to `st.plotly_chart`.
- **Streamlit component API** – `st.set_page_config`, `st.sidebar`, `st.columns`, `st.metric`, `st.dataframe`, `st.markdown(unsafe_allow_html=True)` for custom cards.
- **Google Fonts web API** – typography loaded through CSS `@import`.
- No third-party REST API is called at run time, so the dashboard works offline once fonts are cached.

CHAPTER 7

Features

7.1 List of Main Features

Table 7.1: *Main features of RetailIQ*

#	Feature	Description
1	Automated data preparation	Inspection, quality checks, cleaning and imputation of 7 raw tables
2	Unified master dataset	2,54,331 order lines × 69 columns with 15 engineered features
3	Executive KPIs	Revenue, profit, orders, customers and return rate with trend and insight cards
4	Sales analytics with filters	Date-range and category filters, monthly trend, weekday and quarter views
5	Customer intelligence	RFM segments, K-Means clusters, top customers and segment action plan
6	Product intelligence	Category revenue vs profit, top products, brand table, loss-making SKUs
7	India analytics	State and city rankings, revenue vs profit, state margins
8	Returns & payments	Return reasons/status/category, payment methods, success and failure rates
9	Reviews & experience	Rating distribution, sentiment, category ratings, verified share
10	Premium UI	Dark theme, Indian currency formatting, responsive wide layout, hover tooltips

7.2 How Each Feature Works

7.2.1 Executive Overview

User perspective. The landing page summarises the marketplace in five KPI tiles: **Total Revenue ₹244.99 Cr, Net Profit ₹19.73 Cr (8.1% margin), 1,00,000 orders, 22,820 purchasing customers and a 12.07% return rate**. Below them, a Plotly area chart shows the monthly revenue trend, a bar chart ranks categories and a donut chart shows order status. "Business Pulse" cards state the market leader (Maharashtra), the category leader (Electronics) and the profit watch (33 loss-making products).

Technical description. Technically, the KPIs are computed from the cached feature table with `sum()` and `nunique()`, and the insight text is generated from the top rows of the state, category and product summaries so it updates automatically when the data changes.

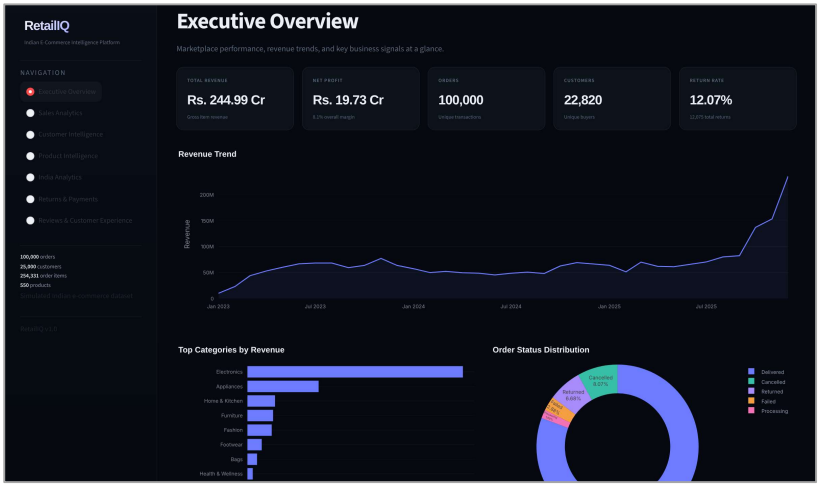


Figure 7.1: *Executive Overview page*

7.2.2 Sales Analytics

User perspective. Users choose an analysis period and categories; every KPI and chart on the page recalculates for that slice – revenue, profit, orders, units sold, the monthly trend, and revenue by weekday and quarter.

Technical description. The page filters the feature DataFrame on the `date` column and selected categories, then aggregates with `groupby` before plotting.

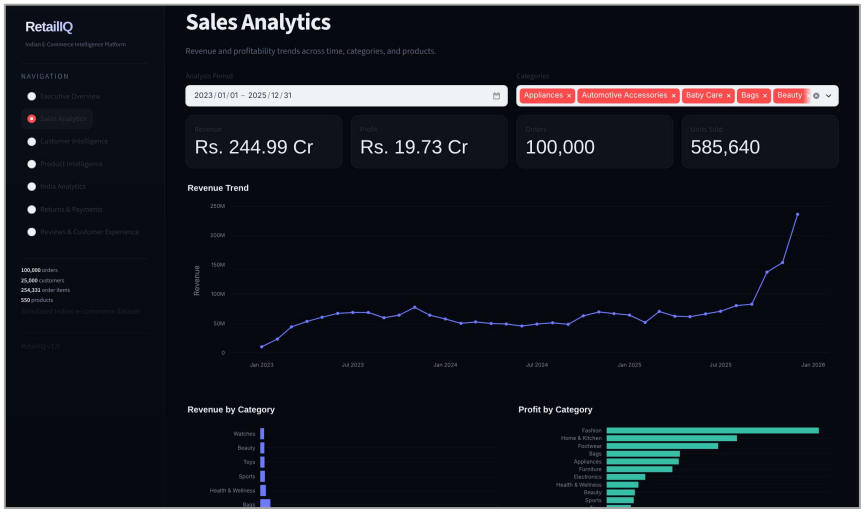


Figure 7.2: *Sales Analytics page*

7.2.3 Customer Intelligence

User perspective. Marketing teams see the number of purchasing customers (22,820), average frequency (4.38 orders) and average spend (₹1.1 L), the size of each RFM segment, a bubble

chart of the two K-Means clusters, the top customers by spend and a recommended action for every segment (e.g. "Win-back campaigns with personalised re-engagement" for At Risk).

Technical description. Data comes from `customer_rfm.csv`, `customer_clusters.csv` and `cluster_summary.csv` produced by the customer modules.

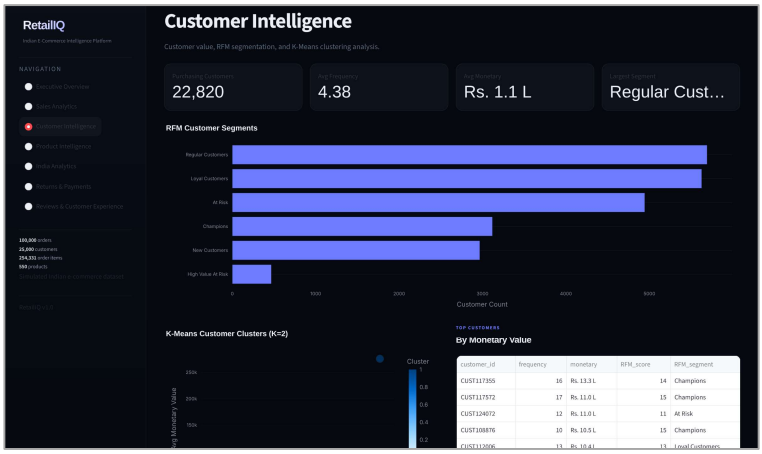


Figure 7.3: *Customer Intelligence page*

7.2.4 Product Intelligence

User perspective. Merchandisers compare categories on revenue against profit, inspect the top products, and review the full category and brand performance tables to spot low-margin or loss-making lines.

Technical description. The scatter of revenue against profit uses margin as colour so that high-revenue/low-margin categories such as Electronics stand out immediately.

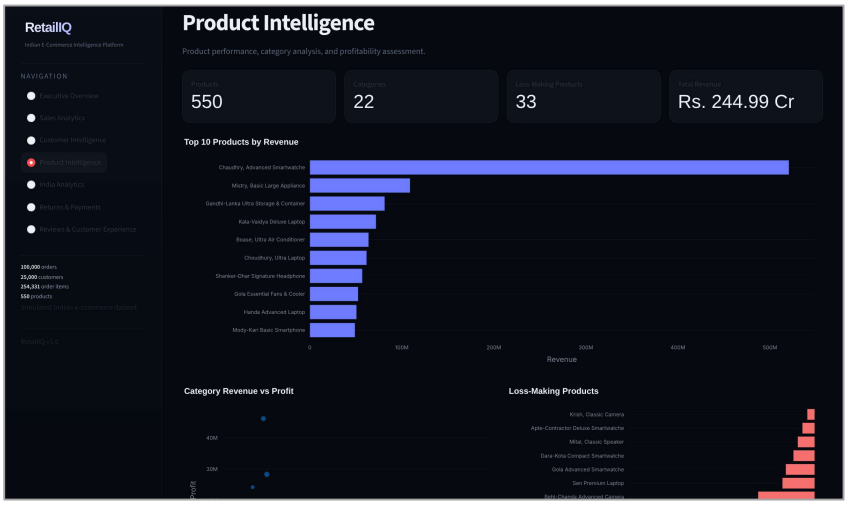


Figure 7.4: *Product Intelligence page*

7.2.5 India Analytics

User perspective. Regional managers see the top 15 states by revenue, a revenue-versus-profit scatter, the highest-margin states and city-level tables.

Technical description. Built from `state_performance.csv` and `city_performance.csv`.

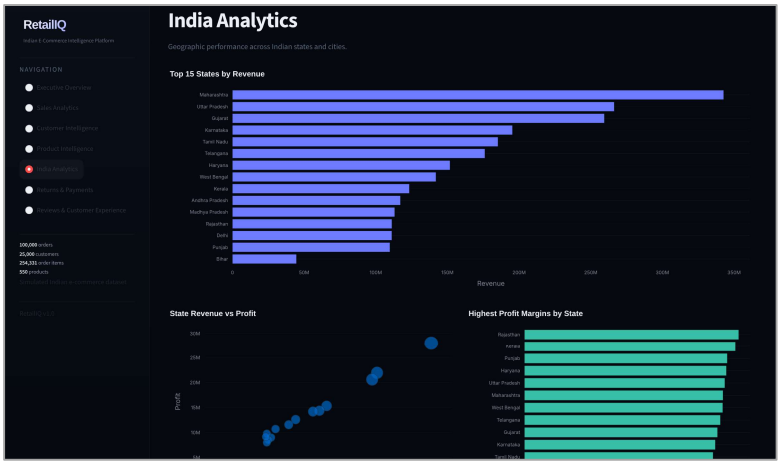


Figure 7.5: *India Analytics page*

7.2.6 Returns & Payments

User perspective. Operations and finance users see total returns, return rate, refund status, return reasons and category return counts, along with the payment-method mix and payment-status distribution.

Technical description. Built from the operations-analysis CSV outputs.



Figure 7.6: *Returns & Payments page*

7.2.7 Reviews & Customer Experience

User perspective. Customer-experience teams see the rating distribution, sentiment donut, top categories by average rating and a customer-experience snapshot (59.7% positive, 9.0% negative, 97.1% verified).

Technical description. Built from rating, sentiment, category-review and verified-review summaries.



Figure 7.7: *Reviews & Customer Experience page*

CHAPTER 8

Testing

RetailIQ has no REST API, so Postman-style endpoint testing does not apply. Testing instead focused on the correctness of each pipeline stage, the consistency of numbers across stages, and the behaviour of the dashboard.

8.1 Functional Testing (Pipeline and Dashboard)

Each script was executed independently and its console output and generated files were checked against expectations: row counts, column counts, absence of nulls in imputed columns, value ranges of scores and margins, and the presence of every output file the dashboard expects. Dashboard pages were exercised manually by switching pages, changing filters and hovering over charts.

8.2 Integration Testing

Integration tests verify that the stages fit together without losing or duplicating data:

- **Referential integrity** – `data_quality.py` confirms zero orders without a customer, zero order items without an order or product, and zero payments, returns or reviews without an order.
- **Join cardinality** – the master table must have exactly as many rows as order_items (2,54,331); a mismatch would indicate duplicated keys.
- **Return flag** – mapping returns as a flag instead of a join keeps row counts unchanged while marking 25,739 order lines (9,510 orders).
- **Cross-stage reconciliation** – total revenue in the dashboard equals the sum of category revenue and the sum of state revenue.
- **Dashboard load** – all 18 files load successfully and every page renders without exceptions.

8.3 Tools Used for Testing

Table 8.1: *Testing tools*

Tool	Use
Python console / print checks	Shape, dtype and value-count assertions after each stage
Pandas assertions	`isna().sum()`, `isin()` checks, `len()` comparisons

Tool	Use
scikit-learn metrics	Silhouette coefficient for clustering quality
Streamlit dev server	Hot-reload manual UI testing in Chrome and Edge
Playwright (headless Chromium)	Automated navigation of all 7 pages and screenshot capture
VS Code debugger	Step-through debugging of transformation logic

8.4 Test Cases and Results

Table 8.2: *Test cases and results*

ID	Test case	Expected result	Actual result	Status
TC01	Load 7 raw CSV files	All load with documented shapes	25,000 / 1,00,000 / 2,54,331 / 550 / 1,00,000 / 12,075 / 77,530 rows	Pass
TC02	Orphan foreign keys	0 orphans in every relation	0 in all 7 checks	Pass
TC03	Impute customer age	No nulls; median 37	250 filled with 37.0	Pass
TC04	Derive missing sentiment	No nulls; rule applied	233 filled from rating	Pass
TC05	Master table rows	= order_items rows	2,54,331 rows × 54 cols	Pass
TC06	Feature table columns	15 new features	69 columns	Pass
TC07	Profit margin infinities	No ±∞ values	Replaced by 0	Pass
TC08	RFM scores range	R, F, M in 1–5	All within 1–5	Pass
TC09	Segment coverage	Every buyer has one segment	22,820 / 22,820	Pass
TC10	K-Means reproducibility	Same silhouettes on re-run	Identical (seed 42)	Pass
TC11	Best K selection	Max silhouette chosen	K = 2, s = 0.4378	Pass
TC12	Revenue reconciliation	Category sum = state sum = KPI	₹244.99 Cr in all three	Pass
TC13	Return rate	12,075 / 1,00,000	12.07%	Pass
TC14	Dashboard navigation	All 7 pages render	Rendered without errors	Pass
TC15	Sales date filter	KPIs change with period	Recalculated correctly	Pass

CHAPTER 9

Deployment

9.1 Steps to Deploy

1. Install Python 3.13 and clone or extract the RetailIQ project folder.
2. Create and activate a virtual environment: `python -m venv .venv` then `.venv\Scripts\activate` (Windows).
3. Install dependencies: `pip install pandas numpy scikit-learn matplotlib plotly streamlit`.
4. Place the seven Kaggle CSV files in `datasets/`.
5. Run the pipeline from the project root in order: `clean_data.py → integrate_data.py → feature_engineering.py → eda.py → customer_analysis.py → customer_clustering.py → product_analysis.py → geography_analysis.py → operations_analysis.py`.
6. Start the dashboard: `streamlit run dashboard/app.py`.
7. Open `http://localhost:8501` in a browser.

9.2 Environment Configuration

Table 9.1: *Environment configuration*

Setting	Value	Where
Working directory	Project root (scripts use relative paths datasets/ and outputs/)	Terminal
Python interpreter	System / venv Python 3.13	.vscode/settings.json
Streamlit page config	Wide layout, expanded sidebar, title "RetailIQ | E-Commerce Intelligence"	app.py
Server port	8501 (default), headless on servers	.streamlit/config.toml or --server.port
Random seed	42 for K-Means	customer_clustering.py
Caching	@st.cache_data on load_data()	app.py

CHAPTER 10

Challenges & Limitations

10.1 Issues Faced During Development

- **Data volume** – the integrated feature table (144 MB, 2.5 lakh rows × 69 columns) made the dashboard slow to start when it was re-read on every interaction.
- **One-to-many joins** – joining returns directly onto order lines duplicated rows and inflated revenue.
- **Different kinds of missing values** – some nulls meant "unknown", others meant "not applicable" (no coupon, never purchased).
- **Skewed RFM features** – monetary values in lakhs overwhelmed order counts in distance-based clustering.
- **Infinite margins** – zero-revenue lines produced ±∞ profit margins.
- **Visual consistency** – default Streamlit and Plotly styling looked inconsistent across 30+ charts.

10.2 Current Limitations and Known Issues

1. **Simulated data** – findings (margins, return rates) reflect the simulation and cannot be generalised to specific real platforms.
2. **Batch processing** – the pipeline works on a fixed snapshot; new data requires re-running the scripts.
3. **No authentication** – the dashboard is intended for local or trusted use only.
4. **Clustering sensitivity** – K-Means results depend on scaling and K and must be re-validated as behaviour changes; with K = 2 the clusters are broad.
5. **Rule-based sentiment** – sentiment is derived from star ratings, not from review text.
6. **Deprecation warnings** – Streamlit reports that `use_container_width` will be replaced by `width`; this does not affect functionality but should be updated.

CHAPTER 11

Future Enhancements

11.1 Planned Features

- **Predictive customer analytics** – churn prediction and Customer Lifetime Value forecasting with supervised models.
- **Sales forecasting** – time-series models (Prophet / ARIMA) for monthly revenue and category demand.
- **Market-basket analysis** – Apriori / FP-Growth association rules for cross-selling recommendations.
- **Text sentiment** – transformer models (e.g. BERT) for aspect-based sentiment once review text is available.
- **Choropleth map of India** – state-level revenue map on the India Analytics page.
- **Downloadable reports** – one-click PDF / Excel export of each page.
- **User login and roles** – authentication with role-based page access.

11.2 Possible Integrations and Optimizations

- **Streaming ingestion** with Apache Kafka or scheduled Apache Airflow DAGs.
- **Data warehouse** – PostgreSQL or Snowflake instead of CSV files; Parquet for faster local I/O.
- **Containerisation and CI/CD** – Docker image built and deployed via GitHub Actions to AWS / GCP.
- **Performance** – Polars or DuckDB for faster aggregation, and lazy loading of page-specific data.
- **Alerting** – email / Slack alerts when return rate or payment failures exceed thresholds.

CHAPTER 12

Conclusion

12.1 Summary of the Project

RetailIQ is an integrated, end-to-end e-commerce analytics platform. It ingests, cleans and integrates seven heterogeneous tables covering 25,000 customers, 1,00,000 orders, 2,54,331 order lines, 550 products, 1,00,000 payments, 12,075 returns and 77,530 reviews; engineers temporal, financial, customer, product, discount and return features; analyses customers, products, sales, geography and operations; and delivers the results through a seven-page interactive Streamlit dashboard.

12.2 What Was Achieved

- A reproducible pipeline producing a 2,54,331 × 69 analytical feature table with 100% referential integrity.
- Six actionable RFM segments and two K-Means clusters, with segment-specific recommendations.
- Evidence that revenue leaders are not profit leaders: Electronics 0.69% margin versus Fashion 33.14%.
- Identification of the main return drivers (size 18.2%, change of mind 16.4%, damage 16.0%) and payment patterns (UPI 32%).
- A polished dashboard that answers the four research questions in one place.

12.3 Skills Learned During Development

Area	Skills
Data engineering	Relational joins, handling missingness, avoiding join duplication, file-based pipelines
Analytics	EDA, KPI design, RFM modelling, profitability and geographic analysis
Machine learning	Feature scaling, K-Means, silhouette-based model selection, reproducibility
Visualization & UI	Plotly, Streamlit, CSS theming, dashboard information design
Software practice	Modular code, Git, testing by reconciliation, documentation and research writing
Soft skills	Team collaboration, time-boxed planning (7-week plan), presenting to reviewers

CHAPTER 13

References

[1] J. Hughes, "Boosting response rates with RFM," *Journal of Database Marketing & Customer Strategy Management*, vol. 3, no. 1, pp. 4–10, 1996.

[2] J. B. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Math. Statist. and Prob.*, vol. 1, pp. 281–297, 1967.

[3] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.

[4] W. Wei, "Customer segmentation model based on RFM and K-means algorithm," *Journal of Physics: Conference Series*, vol. 1871, no. 1, p. 012071, 2021.

[5] C. H. Cheng and Y. S. Chen, "Classifying the segmentation of customer value via RFM model and RS theory," *Expert Systems with Applications*, vol. 36, no. 3, pp. 4176–4184, 2009.

[6] E. W. T. Ngai, L. Xiu and D. C. K. Chau, "Application of data mining techniques in customer relationship management: A literature review and classification," *Expert Systems with Applications*, vol. 36, no. 2, pp. 2592–2602, 2009.

[7] D. Chen, S. L. Sain and K. Guo, "Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining," *Journal of Database Marketing & Customer Strategy Management*, vol. 19, no. 3, pp. 197–208, 2012.

[8] Y. L. Chen and H. H. Huang, "Discovering recency, frequency, and monetary patterns for customer value analysis," *Information Sciences*, vol. 177, no. 11, pp. 2277–2292, 2007.

[9] A. K. Jain, "Data clustering: 50 years beyond K-means," *Pattern Recognition Letters*, vol. 31, no. 8, pp. 651–666, 2010.

[10] L. Kaufman and P. J. Rousseeuw, *Finding Groups in Data: An Introduction to Cluster Analysis*. New York: Wiley, 1990.

[11] S. Few, *Information Dashboard Design: Displaying Data for At-a-Glance Monitoring*, 2nd ed. Burlingame, CA: Analytics Press, 2013.

[12] J. Han, J. Pei and H. Tong, *Data Mining: Concepts and Techniques*, 4th ed. Morgan Kaufmann, 2022.

[13] G. Shmueli, P. C. Bruce, P. Gedeck and N. R. Patel, *Data Mining for Business Analytics: Concepts, Techniques, and Applications in Python*. Hoboken, NJ: Wiley, 2019.

[14] F. Provost and T. Fawcett, *Data Science for Business*. Sebastopol, CA: O'Reilly Media, 2013.

[15] B. Liu, *Sentiment Analysis and Opinion Mining*. Morgan & Claypool, 2012.

[16] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf. (SciPy)*, pp. 56–61, 2010.

[17] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[18] Streamlit documentation, https://docs.streamlit.io

[19] Plotly Python graphing library documentation, https://plotly.com/python/

[20] Pandas documentation, https://pandas.pydata.org/docs/

[21] Kaggle, "E-Commerce Sales and Customer Analytics" (simulated Indian e-commerce dataset), https://www.kaggle.com

CHAPTER 14

Appendices

Appendix A – Screenshots of the Application



Figure 14.1: *Sales Analytics – trend and breakdown charts (scrolled)*

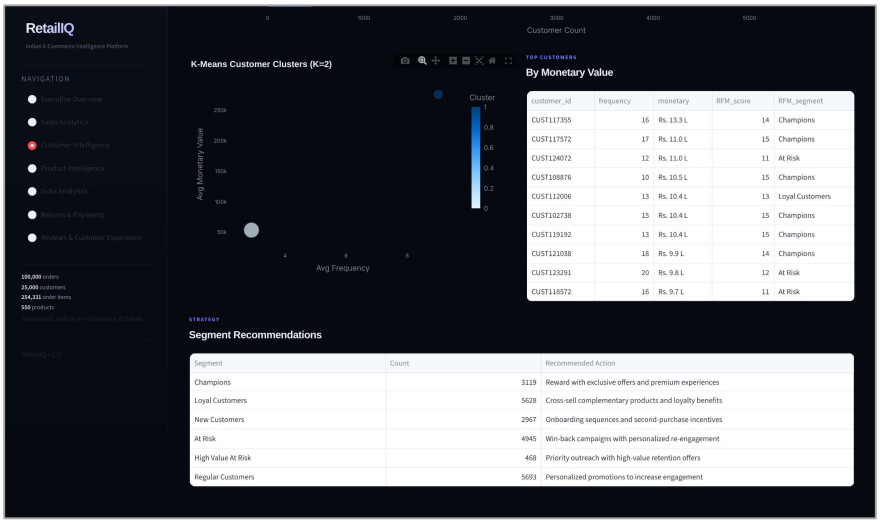


Figure 14.2: *Customer Intelligence – K-Means clusters, top customers and segment recommendations*

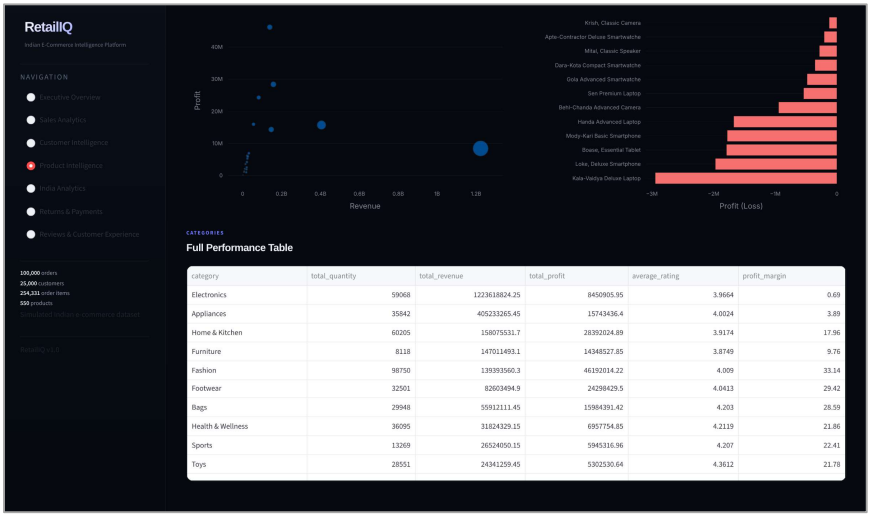


Figure 14.3: *Product Intelligence – category performance table*

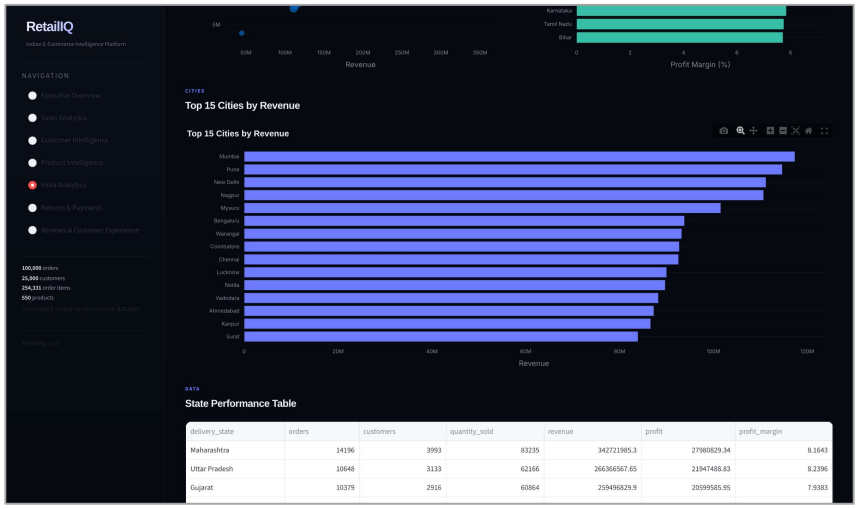


Figure 14.4: *India Analytics – state revenue vs profit and margins*

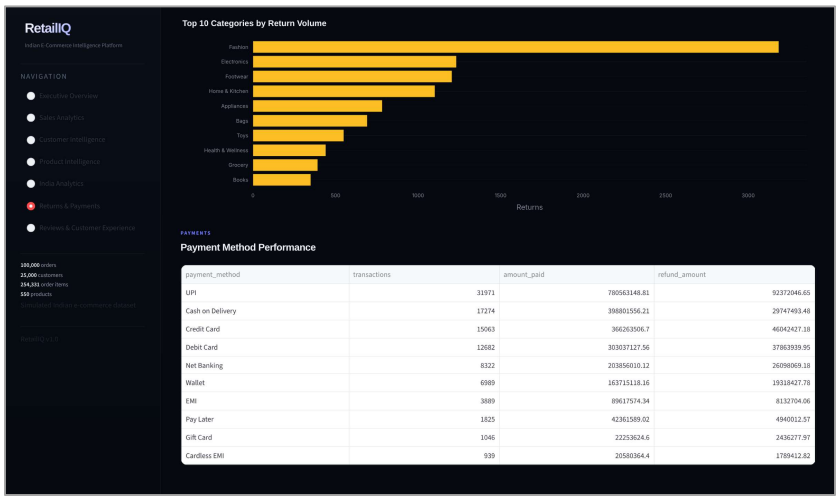


Figure 14.5: *Returns & Payments – payment analysis*

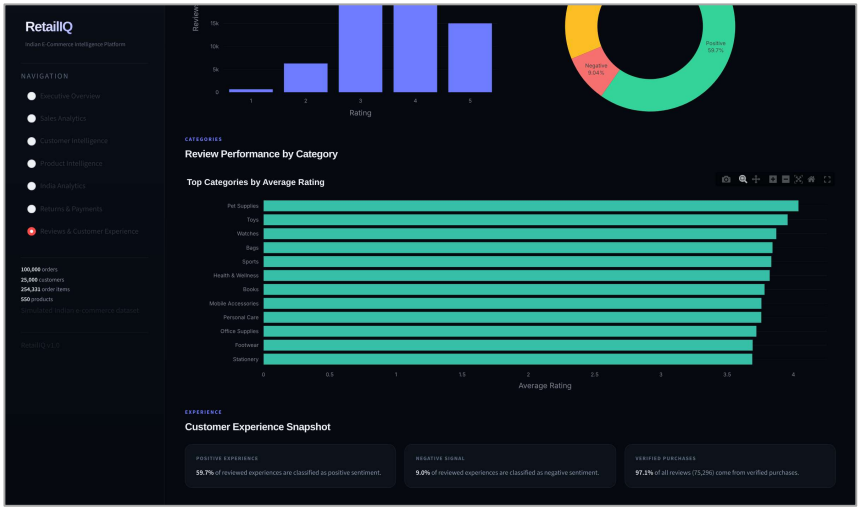


Figure 14.6: *Reviews & Experience – category ratings and experience snapshot*

Appendix B – Sample Code Snippets

Listing B.1 – RFM aggregation

```
# Customer-level RFM aggregation (customer_analysis.py)
customer_data = df.groupby("customer_id").agg(
    recency=("order_date", lambda x: (df["order_date"].max() - x.max()).days),
    frequency=("order_id", "nunique"),
    monetary=("revenue", "sum"),
    total_quantity=("quantity", "sum"),
    average_order_value=("final_amount", "mean"),
    average_profit=("item_profit", "mean"),
).reset_index()
```

Listing B.2 – return flag

```
# Return flag without join duplication (feature_engineering.py)
returns = pd.read_csv("outputs/cleaned/returns_cleaned.csv")
returned_orders = set(returns["order_id"].unique())
df["has_return"] = df["order_id"].apply(lambda x: 1 if x in returned_orders else 0)
```

Listing B.3 – integrity checks

```
# Referential integrity checks (data_quality.py)
print("Orders with missing customers:",
      (~orders["customer_id"].isin(customers["customer_id"])).sum())
print("Order items with missing products:",
      (~order_items["product_id"].isin(products["product_id"])).sum())
print("Returns with missing orders:",
      (~returns["order_id"].isin(orders["order_id"])).sum())
```

Appendix C – Installation and Setup Instructions

Table 14.1: *Installation steps*

Step	Command / action
1. Get the code	Extract RetailIQ.zip (folders: src/, dashboard/, datasets/, outputs/)
2. Python	Install Python 3.13 and confirm with `python --version`
3. Environment	`python -m venv .venv` and activate it
4. Packages	`pip install pandas numpy scikit-learn matplotlib plotly streamlit`
5. Pipeline	Run the scripts in src/ in the order listed in Section 9.1 (skip if outputs/ is already populated)
6. Launch	`streamlit run dashboard/app.py`
7. Open	Browse to http://localhost:8501

Appendix D – User Manual

1. Open the dashboard; the **Executive Overview** page appears with the headline KPIs.
2. Use the **Navigation** list in the left sidebar to switch between the seven pages.
3. On **Sales Analytics**, pick a start and end date and optionally restrict categories; all figures update instantly.
4. Hover over any chart to see exact values; use the Plotly toolbar to zoom, pan, reset or download the chart as PNG.
5. Sort any table by clicking a column header; expand it to full screen with the table toolbar.
6. Read the insight and recommendation cards at the bottom of each page for the key takeaway.
7. To refresh the data, replace the CSVs in `datasets/`, re-run the pipeline, and reload the browser page.

Appendix E – Certifications and Publication Details

A research paper based on this project, **"RetailIQ: An End-to-End E-Commerce Customer and Sales Analytics Platform"** by B. Kundana Mani and M. Sai Kridhay Kumar (Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Hyderabad), has been prepared describing the methodology, experimental setup and results presented in this report.